Национальный исследовательский ядерный университет «МИФИ»

Дискретная математика
Неклассические логические исчисления

Лабораторная работа 1

Верификатор доказательств теорем
Вариант 3

Москва 2026

Цель работы

Разработать консольную программу, которая проверяет каждую вводимую строку доказательства в системе исчисления высказываний. Формула принимается, если она является исходной аксиомой, уже доказана либо получается из доступных формул одним применением modus ponens или одной равномерной подстановкой β.

Результат

Реализована программа на Python 3.10+ без внешних зависимостей для запуска и тестирования. Предусмотрены интерактивный и пакетный режимы, печать дерева, список доказанных строк и очистка сеанса. Для каждого принятого шага сохраняется объяснение с источниками.

Проверка на Python 3.12.14 и 3.14.3 завершилась успешно: 60 автоматических тестов на каждой версии. Демонстрационное доказательство содержит семь принятых строк. Отрицательный набор содержит восемь отклоненных формул.

До реализации изучены 26 исходных файлов: 58 слайдов, пять страниц PDF и 21 изображение. Подробный разбор и перечень контрольных сумм включены в комплект работы.

1 Формальная система варианта 3

Базовые связки — отрицание и дизъюнкция. Переменная обозначается одной латинской буквой. Формула строится из переменной, отрицания формулы или дизъюнкции двух формул. Буквы f и t в этом варианте являются обычными переменными: специальные константы не заданы.

$$A \supset B := \neg A \vee B$$

Импликация используется только как сокращение. Ее раскрытие выполняется при разборе; семантические преобразования дерева не выполняются. Аксиомы трактуются как схемы с согласованными подстановками параметров:

$$A1 \quad p \supset (p \vee q)$$

$$A2 \quad (p \vee p) \supset p$$

$$A3 \quad (p \vee (q \vee r)) \supset (q \vee (p \vee r))$$

$$A4 \quad (q \supset s) \supset ((p \vee q) \supset (p \vee s))$$

$$A5 \quad \neg\neg p \supset p$$

$$A6 \quad (\neg q \supset \neg p) \supset (p \supset q)$$

$$A7 \quad \neg p \vee p$$

В пункте 7 задания после формулы напечатано равенство с другой записью через импликации. В набор включена левая формула ¬p∨p. Знак равенства не входит в язык ввода, правая запись не добавляется как отдельная аксиома. При принятом раскрытии импликации это разные деревья.

Правила вывода

$$MP \quad A, A \supset B \vdash B$$

$$\beta \quad A \vdash [X/x]A$$

В β заменяются все вхождения ровно одной переменной x на одно дерево X. Вставляемая формула может быть составной. Для β замена разных переменных требует нескольких строк. Прямой экземпляр схемы аксиомы принимается сразу, в том числе с несколькими подстановками.

Например, p⊃p раскрывается в ¬p∨p и совпадает с A7. Длинный пример K/S из методички относится к другому варианту и не определяет набор аксиом этой реализации.

2 Синтаксический анализ и дерево

Лексер сохраняет позиции токенов в исходной строке и распознает однобуквенные переменные, операции и скобки. Математические варианты латинских букв из PDF нормализуются по NFKC посимвольно. Пробелы игнорируются, но соседние буквы не объединяются в имя переменной.

Операция	**Запись**
Отрицание	¬, ~, !
Дизъюнкция	∨, |
Импликация	⊃, →, ->
Группировка	Круглые скобки; лишние парные скобки допустимы

Разбор реализован рекурсивным спуском. Грамматика EBNF использует повторение в фигурных скобках и выбор через вертикальную черту:

```
formula     = disjunction { IMP disjunction }
disjunction = unary { OR unary }
unary       = VARIABLE | NOT unary | "(" formula ")"
```

Приоритет убывает от отрицания к дизъюнкции и затем к импликации. Бинарные связки группируются слева. Для импликации это соответствует лекции 1, слайд 16:

$$a \supset b \supset c = (a \supset b) \supset c \neq a \supset (b \supset c)$$

Неизменяемое дерево использует три типа узлов: Var(name), Not(operand), Or(left, right). Узла скобок нет. Например, команда :tree p -> q печатает:

```
OR
  NOT
    p
  q
```

Структурное равенство требует одинаковых операций и одинаковых соответствующих поддеревьев. Перестановка дизъюнктов, иная группировка и удаление двойного отрицания не выполняются. Поэтому логически эквивалентные выражения не обязательно совпадают в верификаторе.

Ошибки содержат позицию: недопустимый символ, отсутствующий операнд, скобка или операция. Для защиты от чрезмерного ввода установлены пределы 4096 символов, 511 узлов после раскрытия и глубина 80.

3 Проверка шага доказательства

Сеанс хранит семь схем аксиом, принятые строки Step и их основания Reason. Источник Reference указывает номер аксиомы или более ранней строки. Доступные деревья индексируются словарем; для MP дополнительно строится индекс по консеквенту. Ошибочная строка не изменяет ни доказательство, ни индексы.

Порядок проверки

1. Разобрать ввод и проверить ограничения. При синтаксической ошибке вернуть отказ.

2. Проверить прямое совпадение со схемой аксиомы и вывести ее номер и подстановки. Затем проверить повтор ранее доказанной строки.

3. Найти в индексе импликацию с консеквентом B и проверить наличие ее антецедента A среди доступных деревьев.

4. Для каждого доступного источника и каждой его переменной проверить, является ли B одной равномерной подстановкой.

5. Если основание найдено, добавить строку с объяснением. Иначе сообщить, что непосредственный шаг не обоснован.

Сопоставление подстановки

Для выбранной переменной x параллельно обходятся источник и результат. При первом вхождении x запоминается соответствующее поддерево X результата. На следующих вхождениях требуется точно такое же X. Все остальные переменные и связки обязаны совпасть. Несогласованность отвергает кандидата. Это позволяет найти вставку без перебора всех возможных формул.

$$[p \vee q / p](p \vee p) = (p \vee q) \vee (p \vee q)$$

Вставленное дерево повторно не обходится. Допускается наличие x в X. Частичная замена x и изменение другой переменной в том же шаге недопустимы. Простая замена подстроки вместо обхода дерева не используется.

Корректность проверки

Индукция по принятым строкам: прямые экземпляры схем разрешены по определению; повтор ссылается на уже установленный факт; MP проверяет обе посылки; сопоставление β строит свидетельство одной равномерной подстановки. Поэтому каждая новая строка имеет разрешенное основание. Будущие и отклоненные строки недоступны.

Алгоритм исчерпывает основания одного шага относительно текущего конечного набора. Он не ищет произвольное многошаговое доказательство. Отказ не утверждает недоказуемость формулы во всей системе. Семантическая эквивалентность и таблицы истинности не используются для приема строк.

4 Демонстрационное доказательство

Файл examples/demo.txt содержит последовательность из семи формул. Первые пять строк — экземпляры схем. Шестая строка получается по MP, седьмая — одной β из предыдущей строки. Аксиомы доступны без предварительного ручного ввода.

$$1.\ \neg q \supset \neg q$$

Экземпляр A7: p := ¬q.

$$2.\ (\neg q \supset \neg q) \supset (q \supset q)$$

Экземпляр A6: p := q; q := q.

$$3.\ q \supset q$$

Экземпляр A7: p := q.

$$4.\ p \supset (p \vee r)$$

Экземпляр A1: p := p; q := r.

$$5.\ (q \supset q) \supset ((q \supset q) \vee r)$$

Экземпляр A1: p := (q ⊃ q); q := r.

$$6.\ (q \supset q) \vee r$$

MP из строк 5 и 3.

$$7.\ (q \supset q) \vee (s \vee \neg s)$$

β из строки 6: r заменяется на (s ∨ ¬s).

Для строки 3 возможен и MP из 2 и 1, но программа приоритетно распознает прямой экземпляр A7. Метка аксиомы не ставится на основании цепочки преобразований: шестая и седьмая строки имеют основания MP и β.

Реальный итог запуска: «Проверено: 7. Принято: 7. Отклонено: 0». Протоколы сохранены в evidence/demo_python312.txt и evidence/demo_python314.txt. Формулы результата печатаются после раскрытия импликаций в базисе ¬ и ∨.

5 Тестирование

Выполнены 60 тестов стандартного unittest на Windows 11 с Python 3.12.14 и Python 3.14.3. Дополнительно проверены компиляция исходников, встроенная демонстрация, ее запуск из файла и отрицательный файл. Команда tools/verify.py сохраняет результаты и коды выхода.

Проверка	**Фактический результат**
Полный набор unittest	60/60 на каждой из двух версий Python
Семь исходных аксиом	Все приняты; все общезначимы в независимом булевом тесте
Демонстрация и тот же файл	7 из 7, код выхода 0
Отрицательный файл	0 из 8, код выхода 1 как ожидается
Ошибки файла и UTF-8	Диагностика и код выхода 2
Подстановка	Сопоставление проверено также на 675 конечных комбинациях
Состояние и ссылки	Отказы не добавляются, сеансы независимы, ссылки ведут назад
Исходные материалы	SHA-256 всех 26 файлов совпали с первоначальным перечнем

Проверки отказов

Из методички проверены лишняя скобка, соседние переменные mn, пропущенная операция между p p и импликация без левого или правого операнда. Корректный пример с лишними парными скобками успешно разбирается. Проверены математические Unicode-буквы, приоритеты и левая группировка.

Логические отрицательные случаи: две замены вместо одной в β производной формулы, замена только части вхождений, отсутствие антецедента MP, попытка получить посылку из заключения, автоматическая перестановка дизъюнктов и добавление неверной строки в доступную базу. Формула a⊃(a∨b) принимается сразу как экземпляр A1. Случай (¬¬¬q⊃¬q)⊃(q⊃¬¬q) распознается как A6, а разные выражения на местах одного параметра отвергаются.

Границы проверки

Независимые булевы оценки служат только дополнительной проверкой аксиом и демонстрации. Они не являются математическим доказательством отсутствия всех программных ошибок. Запуск на другом физическом компьютере, на Linux/macOS и на Python 3.10/3.11/3.13 отдельно не выполнялся. Для проекта не требуется сеть или установленный офисный пакет.

6 Запуск и выводы

Для работы достаточно распаковать комплект и открыть терминал в его корневой папке. В Windows используется команда py -3; при отсутствии Python Launcher — python. Внешние пакеты устанавливать не нужно.

```
py -3 lab1.py
py -3 lab1.py --demo
py -3 lab1.py --file examples/demo.txt
py -3 -m unittest discover -s tests -v
py -3 tools/verify.py
```

Интерактивные команды: :help, :axioms, :list, :tree ФОРМУЛА, :reset, :quit. Пустые строки и комментарии с # пропускаются. В пакетном режиме вводится одна формула на строку; файл читается как UTF-8. Максимум сеанса — 1000 принятых строк, после чего можно начать новый сеанс.

Файл или каталог	Назначение
lab1.py, proofchecker/	Точка входа, дерево, парсер, аксиомы, правила и сеанс
tests/	Тесты синтаксиса, подстановки, MP, состояния и CLI
examples/	Готовые положительный и отрицательный наборы
docs/	Разбор материалов, соответствие требованиям, сценарий защиты
evidence/	Фактические протоколы и машиночитаемые итоги

Вывод

Задача выполнена: создан верификатор доказательств для варианта 3 с абстрактным синтаксическим деревом, структурным сравнением, MP и атомарным β. Обоснованные формулы сохраняются с источниками, ошибочный ввод и необоснованные шаги отвергаются. Логика программы отделена от консольного интерфейса и проверяется автоматическими тестами.

Использованные материалы

1. Лабораторная работа №1.pdf, с. 1–3, требования и вариант 3.

2. Методический материал к ЛР.pdf, с. 1–2, примеры синтаксиса и сеанса.

3. Лекция 1.pptx, 25 слайдов, формулы, доказательства, MP, подстановка.

4. Лекция 2-1.pptx, 20 слайдов, выводимость, семантика, многозначные логики.

5. Семинар №1 2024.pptx, 13 слайдов; пять PNG и 16 фотографий доски.